PROJECT TITLE	**:**	JSON Refiner
TEAM ID	**:**	69a9290ffe45d7d8dad414d7
TEAM MEMBERS	**:**	UMA MAHESWARI V VIJAYA POORANI R VIMALA SRI DEVI.A SOBIKA P
COLLEGE NAME	**:**	S. A. V SAHAYA THAI ARTS AND SCIENCE (WOMEN) COLLEGE
COLLEGE CODE	**:**	138
DATE	**:**	07.03.2026

JSON Refiner Advanced Edition

Professional JSON Processing Tool with Gradio Interface

PROJECT DESCRIPTION:

The JSON Refiner project is a Python-based application designed to convert unstructured key-value text into a well-structured JSON format. The system accepts simple text input such as “name: John, age: 28” and automatically converts it into valid JSON by identifying the correct data types such as strings, integers, floats, booleans, and null values.

The project is developed using Python in **Google Colab** and provides an interactive user interface built with **Gradio**, allowing users to easily input data and view the generated JSON output. The system performs automatic type inference, key normalization, and structured formatting to ensure the output follows standard JSON syntax.

The goal of the project is to simplify data processing and make it easier to convert raw textual data into machine-readable JSON format that can be used in APIs, databases, and other software systems. By combining Python’s data processing capabilities with an interactive interface, the JSON Refiner application demonstrates how structured data transformation can be performed efficiently in a user-friendly environment.

.

SCENARIOS

Scenario 1: Converting Raw Text to JSON

A user has unstructured text data written in key-value format such as `name: John, age: 28`. The JSON Refiner system processes this input and automatically converts it into a structured JSON format. This helps users quickly transform raw text data into machine-readable JSON.

Scenario 2: Data Preparation for APIs

Developers often need to send data to APIs in JSON format. Using the JSON Refiner tool, users can enter simple key-value text and convert it into valid JSON that can be directly used in API requests or responses.

Scenario 3: Configuration File Conversion

System administrators sometimes store configuration values in simple text format. The JSON Refiner application can convert these configuration values into structured JSON, making them compatible with modern applications and deployment systems.

Scenario 4: Data Cleaning and Formatting

When working with datasets, users may have inconsistent data formats. The JSON Refiner tool automatically detects data types such as numbers, booleans, and strings, ensuring that the final JSON output is clean, properly formatted, and ready for further processing.

ARCHITECTURE OVERVIEW

The JSON Refiner system is designed as a simple and modular architecture that combines Python-based data processing with an interactive user interface. The system accepts raw key-value text input from the user and processes it through a series of steps including parsing, type inference, and JSON formatting. The processed data is then displayed as structured JSON output.

The backend logic of the application is implemented using the **Python** programming language, which performs tasks such as data parsing, type detection, and JSON generation. The application is developed and executed in **Google Colab**, which provides a cloud-based development environment.

For the user interface, the system uses **Gradio**, which enables users to interact with the application through a simple web interface. Users can enter key-value data, run the processing function, and instantly view the generated JSON output.

The architecture ensures a smooth flow of data from user input to JSON generation while maintaining simplicity, accuracy, and ease of use.

Core Technologies

- **Python** – Used as the main programming language to implement the logic for parsing key-value data, detecting data types, and generating structured JSON output.
- **Google Colab** – Provides a cloud-based development environment where the Python code is written, executed, and tested without requiring local installation.
- **Gradio** – Used to build an interactive web interface that allows users to enter key-value input and instantly view the generated JSON output.
- **JSON Library (Python)** – Used for converting processed data into properly formatted JSON structure

PRE-REQUISITES

Software Requirements

- **Python** – Required for implementing the core logic of the application.
- **Google Colab** – Used as the development environment to write and execute Python code.
- **Gradio** – Required for building the interactive user interface.
- Internet Connection: Required for Colab and model access

Knowledge Prerequisites

- To Basic understanding of Python programming.
- Basic knowledge of JSON format and structure.
- Familiarity with running code in Google Colab.

Hardware Requirements

To develop and run the JSON Refiner project, the following basic hardware requirements are needed:

- **Processor:** Any modern processor such as Intel i3 or equivalent
- **RAM:** Minimum 2 GB RAM (Google Colab provides cloud computing resources)
- **Storage:** At least 500 MB free storage space
- **Display:** Minimum screen resolution of 1024 × 768
- **Internet Connection:** Required to access **Google Colab** and run the application online.

These hardware requirements are minimal since most of the processing is handled through cloud resources.

PROJECT WORKFLOW

The JSON Refiner project follows a structured workflow to convert unstructured key-value text into properly formatted JSON data.

Phase 1: Environment Setup & Model Configuration

The development environment is prepared using **Google Colab**. Required Python libraries such as **Gradio** are installed to support the user interface and data processing features.

MILESTONE 1: Environment Setup & Core Architecture

Activity 1.1: Install Dependencies

The project environment was prepared using **Google Colab**. Required libraries were installed to support the JSON processing and user interface development.

Coding Implementation

```
!pip install gradio json5 jsonschema python-dotenv pydantic
```

Explanation

In this step, the required Python libraries were installed and imported. These libraries enable the application to process JSON data and create an interactive interface for the user.

Activity 1.2: Initialize Core Class

A core Python class was created to handle JSON refinement operations such as parsing input data, detecting data types, and converting key-value pairs into JSON format.

```
class JSONRefinerCore:

    def __init__(self):
        self.conversion_log = []
        self.validation_errors = []
        self.transformation_history = []

    def infer_type(self, value):

        value = value.strip()

        if value.lower() in ["true","yes","on"]:
            return True

        if value.lower() in ["false","no","off"]:
            return False

        if value.lower() in ["null","none","nil"]:
            return None

        try:
            if "." in value:
                return float(value)
            return int(value)
        except:
            pass

        return value
```

Explanation

The core class acts as the main component of the system. It stores transformation logs, validation results, and handles the main logic for processing the input data.

MILESTONE 2: Type Inference & Case Conversion System

Activity 2.1: Implement Type Inference

The system automatically detects the data types of input values such as integers, floating-point numbers, booleans, and strings.

Coding Implementation

```python
class JSONRefinerCore:

    def __init__(self):
        self.conversion_log = []
        self.validation_errors = []
        self.transformation_history = []

    def infer_type(self, value):

        value = value.strip()

        if value.lower() in ["true","yes","on"]:
            return True

        if value.lower() in ["false","no","off"]:
            return False

        if value.lower() in ["null","none","nil"]:
            return None

        try:
            if "." in value:
                return float(value)
            return int(value)
        except:
            pass

        return value
```

Explanation

The type inference logic analyzes each value provided in the input text and determines the appropriate data type before converting it into JSON format.

.

Activity 2.2: Case Conversion Methods

The project supports multiple naming conventions for JSON keys such as snake_case, camelCase, kebab-case, and PascalCase.

Coding Implementation

```python
def convert_to_json(input_text):
    result = key_value_to_json(input_text)
    return json.dumps(result, indent=4)

interface = gr.Interface(
    fn=convert_to_json,
    inputs=gr.Textbox(lines=10, placeholder="Enter key-value pairs"),
    outputs="code",
    title="JSON Refiner",
    description="Convert key-value text into JSON"
)
interface.launch()
```

Explanation

These conversion methods ensure consistent naming conventions for JSON keys, which improves readability and compatibility with different programming standards.

MILESTONE 3: Validation & Transformation Features

Activity 3.1: JSON Parsing

The system processes key-value input data and converts it into a structured JSON format.

Explanation

The parsing function reads the input text line by line, extracts keys and values, and prepares them for JSON conversion.

MILESTONE 4: Gradio Interface Development

Activity 4.1: User Interface Design

A user-friendly interface was created using **Gradio** to allow users to interact with the application.

Coding Implementation

Explanation

The interface allows users to input key-value data and instantly view the generated JSON output.

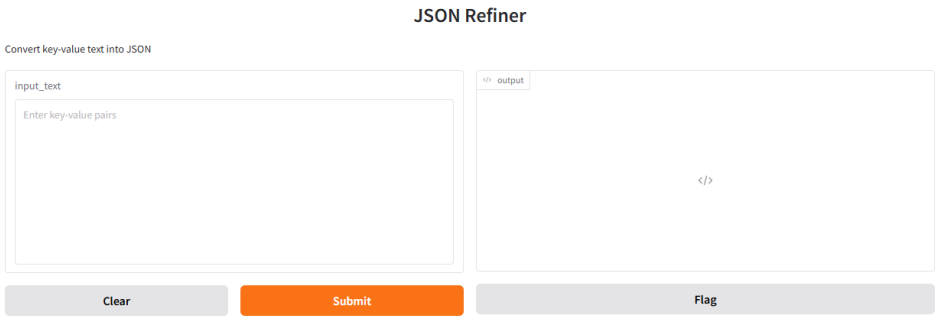
MILESTONE 5: Testing & Output

Activity 5.1: Running the Application

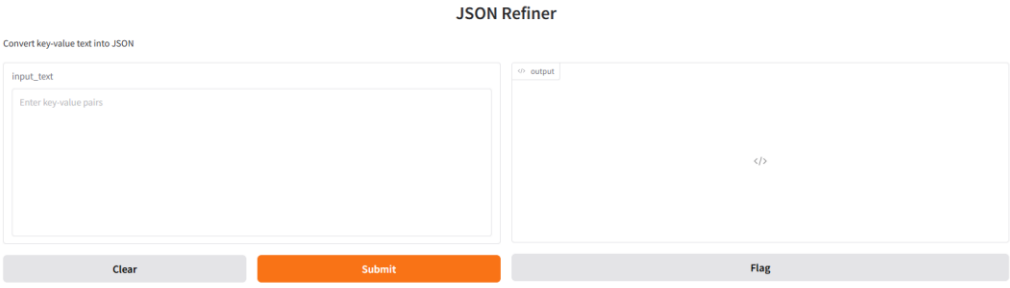
The application was executed in **Google Colab** and tested with sample input data.

Coding Execution

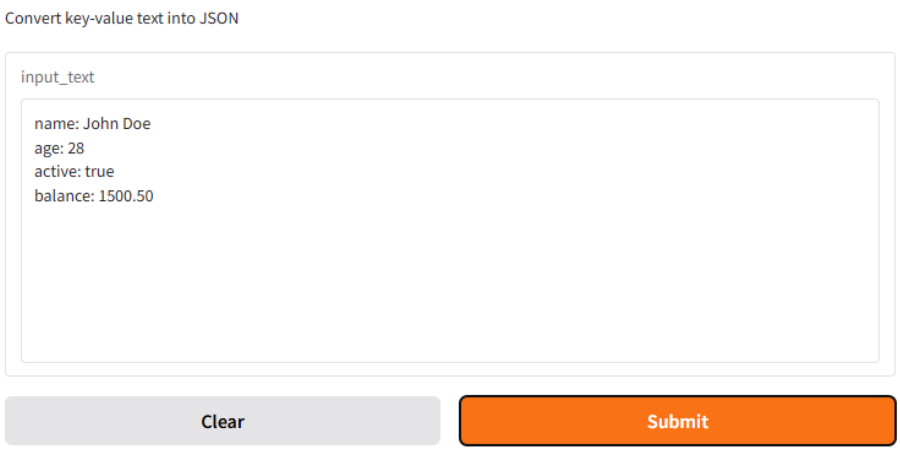
```
It looks like you are running Gradio on a hosted Jupyter notebook, which requires `share=True`. Automatically setting `share=True` (you can turn this off by set

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://5327e4c61999ab6300.gradio.live

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the working directory to deploy to Hugging Face Spaces (htt
```



Output Result



Input Example



Output Example

output

```
{
    "name": "John Doe",
    "age": 28,
    "active": true,
    "balance": 1500.5
}
```

Flag

Explanation

The system successfully converted the raw key-value input data into structured JSON format. The output demonstrates the correct identification of data types and proper JSON formatting.

Conclusion

The JSON Refiner project successfully demonstrates how unstructured key-value text data can be converted into structured JSON format using **Python**. The system processes input data, automatically detects appropriate data types such as strings, integers, floats, and booleans, and generates valid JSON output.

The project was developed and tested using **Google Colab**, which provides a flexible cloud-based environment for executing Python code. An interactive user interface was implemented using **Gradio**, allowing users to easily input key-value data and instantly view the resulting JSON structure.

Overall, the project simplifies data transformation and demonstrates how modern tools can be used to build efficient and user-friendly applications for processing structured data.